

UserAPI — DevOps Project

Python · Flask · PostgreSQL · Docker · Kubernetes · Vagrant · Ansible

Author: Eshwar reddy · August 2026

1. Work Performed

A brief summary of all features and components implemented in this project.

1.1 User API — Core Application

RESTful CRUD API built with Python 3 and Flask, storing user records in PostgreSQL via SQLAlchemy ORM.

- Full CRUD: POST /users · GET /users · GET /users/<id> · PUT /users/<id> · DELETE /users/<id>
- Health check: GET /health — actively queries the DB, returns {"status":"ok","database":"healthy"}
- Input validation: required fields, email format, duplicate username/email (409 / 422 responses)
- Web dashboard: index.html — shows live user count, DB status, API status, Add/Delete user UI
- Application Factory pattern: create_app(config) separates test, dev and production configs cleanly

1.2 Test Suite — Four Levels (46 tests)

File	Type	Tests	What it covers
test_unit.py	Unit	6	User.validate(), to_dict(), __repr__ — pure logic, no HTTP or DB
test_config.py	Config	11	Config class values, env-var overrides, app-level config injection
test_connection.py	Connection	7	DB reachability, schema, insert/delete/rollback/unique constraints
test_api.py	API	22	Full HTTP cycle for every endpoint — 201/200/404/409/422/400

1.3 Docker & Docker Compose

- Dockerfile: python:3.11-slim, gunicorn WSGI, 2 workers, PYTHONPATH wired correctly
- docker-compose.yaml: three services — postgres:16 (5432), pgadmin4 (5050), Flask app (3000)
- Health checks: Compose waits for pg_isready before starting the app
- Named volumes: postgres_data and pgadmin_data persist across container restarts
- Image published to Docker Hub: kousiksc/userapi:latest and eshwarreddy/userapi:latest

1.4 Infrastructure as Code — Vagrant + Ansible

- Vagrantfile: ubuntu/jammy64, port-forwards 3000→3000 and 5432→5433, syncs userapi/ into VM
- Ansible role python: installs python3, pip3, venv, libpq-dev, build-essential
- Ansible role postgresql: creates userapi_db, sets md5 auth, configures listen_addresses
- Ansible role app: installs deps, creates systemd service (userapi.service), runs /health check

1.5 Kubernetes — Minikube

- deployment.yaml: postgres (1 replica + readinessProbe) · userapi (2 replicas + liveness & readiness on /health)
- service.yaml: postgres-service (ClusterIP:5432) · userapi-service (NodePort:30080)
- persistent-volume.yaml + persistent-volume-claim.yaml: 1 Gi hostPath storage for PostgreSQL

- Deployed using Minikube with Docker driver on Windows 11

1.6 CI/CD — GitHub Actions

- Pipeline file: `.github/workflows/ci.yml` — triggers on push and pull_request to main only
- Runner: ubuntu-latest (1× billing multiplier); pip cache enabled — avoids re-downloading packages
- Steps: checkout → setup-python 3.12 → install → unit → config → connection → API tests
- Tests use SQLite :memory: via TestConfig — no PostgreSQL container needed in CI

2. Screenshots

2.1 Docker Compose — All Containers Up

Running `docker-compose up -d` starts all four resources: network, PostgreSQL (Healthy), pgAdmin, and the Flask app.

```
E:\PROJECT2>docker-compose up -d
[+] up 4/4
✓ Network project2_default Created 3.4s
✓ Container userapi_postgres Healthy 63.3s
✓ Container userapi_pgadmin Started 55.6s
✓ Container userapi_app Started 56.2s
```

Fig 1 — `docker-compose up -d`: network + 3 containers all healthy/started

2.2 Docker Desktop — Images

Docker Desktop showing the three pulled images: `project2-app` (Flask), `dpape/pgadmin4`, and `postgres:16`.

☐	☒	project2-app	latest		cf0127f081eb		3 days ago	263.01 MB			
☐	☒	dpape/pgadr	latest		2f4ce946ddf8		7 days ago	777.84 MB			
☐	☒	postgres	16		33f923b05f64		20 days ago	641.57 MB			

Fig 2 — Docker Desktop images: `app` (263 MB), `pgAdmin` (778 MB), `postgres` (642 MB)

2.3 UserAPI Dashboard — Running via Docker Compose

The web dashboard served by `unicorn` at `http://localhost:3000`. Shows 2 users, database Connected, API Running.

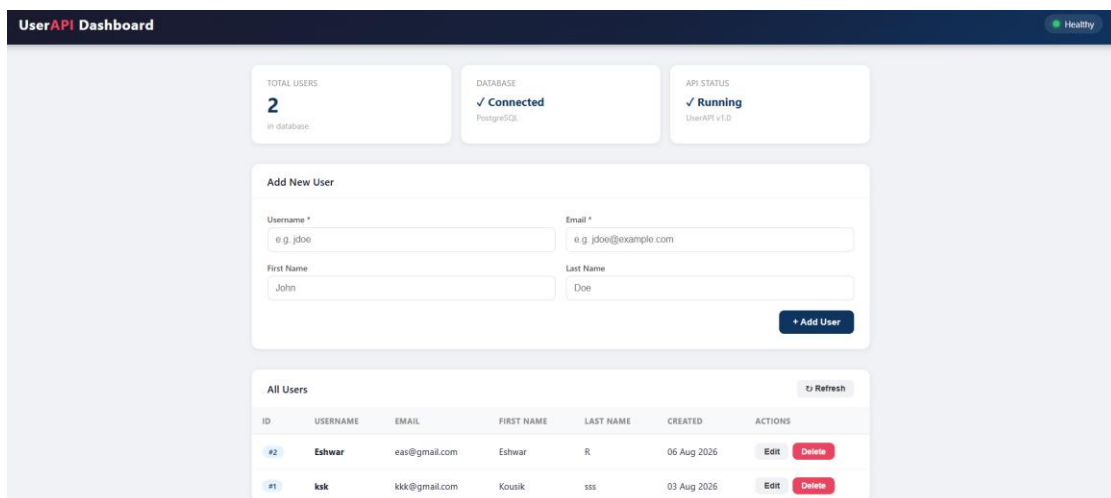


Fig 3 — UserAPI Dashboard at `localhost:3000` — 2 users in DB, Healthy badge active

2.4 pgAdmin — userapi_db Database

pgAdmin 4 connected to PostgreSQL, showing the userapi_db database with its public schema.

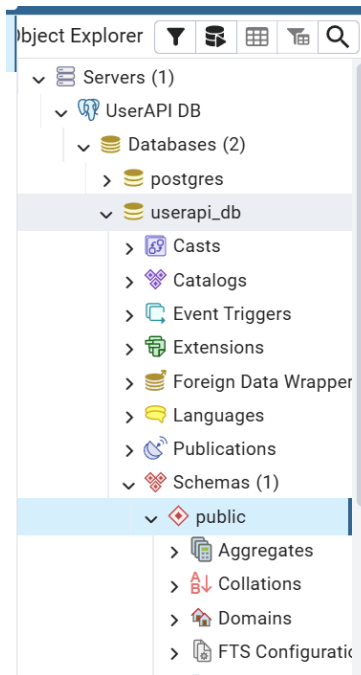


Fig 4 — pgAdmin Object Explorer: UserAPI DB → userapi_db → public schema

2.5 Docker Hub — Published Images

The Docker image was built and pushed to Docker Hub under

[eshwarreddy/userapi:latest](https://hub.docker.com/repository/registry-1.docker.io/eshwarreddy/userapi/latest)

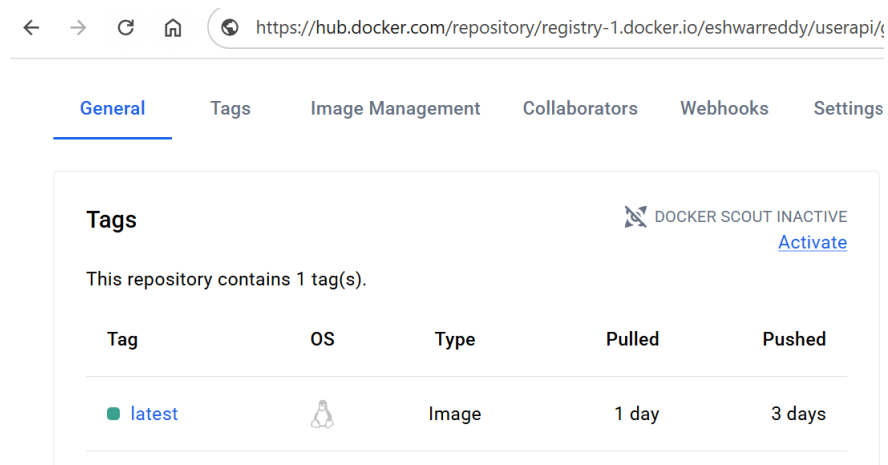


Fig 6 — hub.docker.com/r/eshwarreddy/userapi — latest tag, Linux image, pushed 3 days ago

2.51 CI CD -Github actions

The CI CD pipeline is configured using github actions

✓

CI — all tests passing

passing

push to main · run #4 · 1 job

47s

ubuntu-latest (1× rate)

STEPS · test (ubuntu-latest)

✓ Set up job	2s
✓ Checkout code	checkout@v41s
✓ Set up Python 3.12	setup-python@v54s
✓ Restore pip cache	cache hit1s
✓ Install dependencies	6s
✓ Unit tests	6 passed8s
✓ Configuration tests	11 passed7s
✓ Connection tests	7 passed7s
✓ API tests	22 passed9s
✓ Complete job	2s

46 tests passed · 0 failed · Total: 47s · 1× billing rate

.github/workflows/ci.yml

Save this file in your repository to activate the workflow

ci.yml57 lines

```
1 # .github/workflows/ci.yml - triggers only on main to reduce billing
2
3 name: CI
4
5 on:
6   push:
7     branches: [main]
8   pull_request:
9     branches: [main]
10
11 jobs:
12   test:
13     # ubuntu = 1x minute multiplier (cheapest runner)
14     runs-on: ubuntu-latest
15
16     # No postgres service - TestConfig uses SQLite :memory:
17
18     steps:
19       - name: Checkout code
20         uses: actions/checkout@v4
21
22       - name: Set up Python 3.12
23         uses: actions/setup-python@v5
24         with:
25           python-version: "3.12"
26           cache: "pip" # skips re-download
27
28       - name: Install dependencies
29         run: pip install -e
```

2.6 Vagrant — App Running Inside VM

Inside the provisioned Ubuntu 22.04 VM (vagrant ssh). The app package is installed and the systemd service started.

```
vagrant@userapi-vm:~$ cd ~/userapi
vagrant@userapi-vm:~/userapi$ pip install -e . 2>/dev/null
Defaulting to user installation because normal site-packages is not writeable
Obtaining file:///home/vagrant/userapi
  Preparing metadata (setup.py) ... done
Installing collected packages: userapi
  Attempting uninstall: userapi
    Found existing installation: userapi 1.0.0
    Uninstalling userapi-1.0.0:
      Successfully uninstalled userapi-1.0.0
  Running setup.py develop for userapi
Successfully installed userapi-1.0.0
vagrant@userapi-vm:~/userapi$ sudo systemctl start userapi
vagrant@userapi-vm:~/userapi$
```

Fig 7 — vagrant@userapi-vm: pip install -e . succeeded, sudo systemctl start userapi

2.7 Kubernetes — Minikube Start

Starting Minikube v1.38.1 on Windows 11 using the Docker driver.

```
E:\PROJECT2>minikube start
🤗 minikube v1.38.1 on Microsoft Windows 11 Home Single
Language 25H2
🌟 Using the docker driver based on existing profile
👍 Starting "minikube" primary control-plane node in "mi
nikube" cluster
```

Fig 8 — minikube start: v1.38.1 on Windows 11 Home, docker driver, control-plane booting

2.8 Kubernetes — Exposing the Service

Running minikube service userapi-service --url to get the NodePort URL for the deployed app.

```
E:\PROJECT2>minikube service userapi-service --url
http://127.0.0.1:56996
! Because you are using a Docker driver on windo
ws, the terminal needs to be open to run it.
```

Fig 9 — minikube service userapi-service --url → http://127.0.0.1:56996

2.9 Kubernetes — App in Browser

UserAPI Dashboard accessed via the Minikube NodePort URL (127.0.0.1:56996). DB Connected, API Running.

← → 127.0.0.1:56996 ☆

UserAPI Dashboard

TOTAL USERS
1
in database

DATABASE
✓ Connected
PostgreSQL

API STATUS
✓ Running
UserAPI v1.0

Add New User

Username *
e.g. jdoe

Email *
e.g. jdoe@example.com

First Name
John

Last Name
Doe

+ Add User

Fig 10 — App running on Minikube at 127.0.0.1:56996 — 1 user, Healthy

3. Instructions

3.1 Prerequisites

Tool	Purpose
Python 3.10+	Running tests locally, local dev
Docker Desktop	docker-compose up
VirtualBox 7.x	Required by Vagrant
Vagrant 2.x	vagrant up (IaC provisioning)
Minikube	Kubernetes local cluster
kubectl	Applying K8s manifests

3.2 Environment Setup

Create E:\PROJECT2\userapi\.env:

```
DB_HOST=postgres
DB_PORT=5432
DB_NAME=userapi_db
DB_USER=<user>
DB_PASSWORD=<password>
SECRET_KEY=<secret-key>
```

Note: Inside Docker Compose, DB_HOST=postgres is set automatically via docker-compose.yaml environment block.

3.3 Run with Docker Compose

```
cd E:\PROJECT2

# Build and start all 3 services
docker-compose up --build

# Or detached (background):
docker-compose up -d

# Access:
#   App      → http://localhost:3000
#   Health   → http://localhost:3000/health
#   pgAdmin  → http://localhost:5050   (admin@admin.com / admin)

# Stop and remove containers
docker-compose down
```

3.4 Use the API

```
# Create a user
curl -X POST http://localhost:3000/users \
  -H "Content-Type: application/json" \
  -d '{"username": "alice", "email": "alice@example.com", "firstname": "Alice", "lastname": "Smith"}'
```

```
# List all users
curl http://localhost:3000/users

# Get one user by ID
curl http://localhost:3000/users/1

# Update a user
curl -X PUT http://localhost:3000/users/1 \
  -H "Content-Type: application/json" \
  -d '{"firstname": "Alicia"}'

# Delete a user
curl -X DELETE http://localhost:3000/users/1

# Health check
curl http://localhost:3000/health
```

3.5 Run Tests

```
cd E:\PROJECT2\userapi

# All 46 tests
pytest test/ -v

# By category:
pytest test/test_unit.py -v # 6 unit tests
pytest test/test_config.py -v # 11 config tests
pytest test/test_connection.py -v # 7 connection tests
pytest test/test_api.py -v # 22 API tests
```

3.6 Vagrant (IaC)

```
cd E:\PROJECT2\iac

# First boot - provisions the full Ubuntu VM (~5-10 min)
vagrant up

# SSH into the VM
vagrant ssh

# Inside VM: check app service
sudo systemctl status userapi

# App available at http://localhost:3000 (port-forwarded)

# Stop VM (keeps all data)
vagrant halt

# Resume a halted VM
vagrant up
```

```
# Destroy VM completely
vagrant destroy
```

3.7 Kubernetes (Minikube)

```
# Start Minikube
minikube start

# Apply manifests (from E:\PROJECT2)
kubectl apply -f k8s/persistent-volume.yaml --validate=false
kubectl apply -f k8s/persistent-volume-claim.yaml --validate=false
kubectl apply -f k8s/deployment.yaml --validate=false
kubectl apply -f k8s/service.yaml --validate=false

# Check status
kubectl get pods
kubectl get services
kubectl get pv,pvc

# Open app in browser
minikube service userapi-service --url

# Cleanup
kubectl delete -f k8s/
minikube stop
```

4. Platforms & Links

Platform	URL / Reference	Notes
Docker Hub (kousiksc)	hub.docker.com/r/eshwarreddy/userapi	eshwarreddy/userapi:latest — used in K8s deployment
Docker Hub (eshwarreddy)	hub.docker.com/r/eshwarreddy/userapi	eshwarreddy/userapi:latest
GitHub Repository	github.com/ eshwarreddy	Source code and commit history
GitHub Actions	github.com/ eshwarreddy → Actions	CI pipeline — all tests on every push to main
App (Docker Compose)	http://localhost:3000	Flask app served by gunicorn
pgAdmin (Docker)	http://localhost:5050	admin@admin.com / admin
App (Kubernetes)	minikube service userapi-service --url	NodePort — URL generated by Minikube
App (Vagrant)	http://localhost:3000	Port-forwarded from VM (vagrant up in iac/)
Vagrant Box	app.vagrantup.com/ubuntu/boxes/jammy64	ubuntu/jammy64 v20241002.0.0

5. Author

Name	
Email	
GitHub	
Docker Hub	hub.docker.com/u/ eshwarreddy
Project date	August 2026

6. Additional Information

Project Structure

```
E:\PROJECT2\
├── userapi\                               Flask application
│   ├── src\
│   │   ├── __init__.py                   App factory (create_app)
│   │   ├── app.py                       Entry point
│   │   ├── config.py                   Config / TestConfig / ProductionConfig
│   │   ├── models.py                   User SQLAlchemy model
│   │   ├── routes.py                   All API blueprints & health check
│   │   └── templates\
│   │       └── index.html               Web dashboard
│   ├── test\
│   │   ├── conftest.py                 Fixtures: app, db, client, clean_db
│   │   ├── test_unit.py                Unit tests (6)
│   │   ├── test_api.py                 API tests (22)
│   │   ├── test_config.py              Config tests (11)
│   │   └── test_connection.py           Connection tests (7)
│   ├── Dockerfile
│   ├── requirements.txt
│   └── .env
├── iac\                                   Vagrant + Ansible IaC
│   ├── Vagrantfile
│   └── playbooks\
│       ├── setup.yml
│       └── roles\ (python / postgresql / app)
├── k8s\                                   Kubernetes manifests
│   ├── deployment.yaml                 postgres (1) + userapi (2 replicas)
│   ├── service.yaml                   ClusterIP + NodePort
│   ├── persistent-volume.yaml
│   └── persistent-volume-claim.yaml
├── .github\workflows\
│   └── ci.yml                         GitHub Actions CI pipeline
└── docker-compose.yaml
```

Key Design Decisions

- SQLite for tests: TestConfig switches to `sqlite:///memory:` — no PostgreSQL dependency in CI
- Application Factory: `create_app(config)` lets the same code run under test, dev, and prod configs

- Systemd in Vagrant: app runs as a proper service — auto-restarts on crash, survives reboots
- Gunicorn everywhere: consistent production WSGI server in both Docker and Vagrant
- K8s probes: liveness and readiness both hit /health — Kubernetes restarts pods if DB goes down
- 2 replicas in K8s: demonstrates horizontal scaling — traffic load-balanced across both pods
- Billing-aware CI: triggers only on main, uses pip cache, avoids matrix — preserves free tier minutes

UserAPI DevOps Project Eshwar reddy · August 2026